

NT219 - Cryptography

Week 06: Modern Asymmetric Cryptography (P1)

PhD. Ngọc-Tu Nguyen
tunn@uit.edu.vn

UIT-VNUHCM

October 14, 2025

Outline

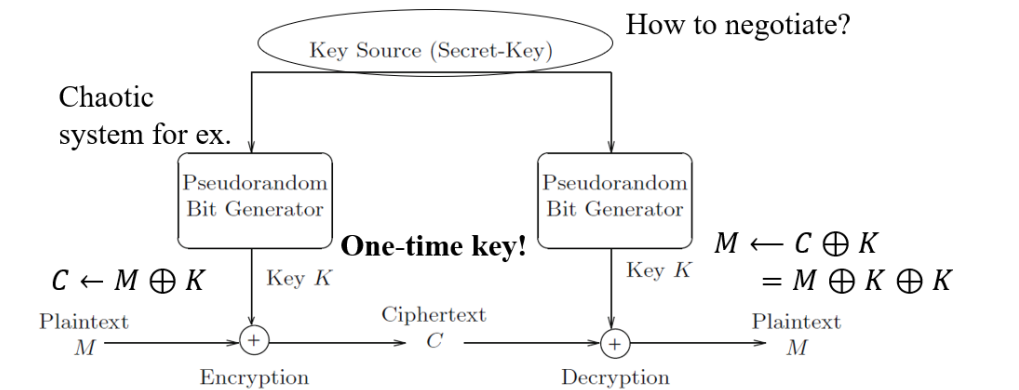
- 1 Motivation and Review
- 2 Introduction to Asymmetric Cryptography
- 3 Public-Key Concepts and Requirements
- 4 Applications of Public-Key Systems
- 5 RSA Algorithm
- 6 Summary

Stream Cipher Review

- Revisit symmetric ciphers and chaotic systems.
- Example: Stream ciphers that generate key streams based on initial seeds.
- Attacks: chosen-plaintext attacks exploit key reuse.
- Recall: One-time key provides perfect secrecy.
- Case study: ChaCha20-Poly1305.

Thuật ngữ: stream cipher = mật mã dòng; chaotic system = hệ hỗn loạn; one-time key = khóa dùng một lần

Stream Cipher Review



Attacks (chosen plaintext attacks): *Known* (M, C) : $C \oplus M = M \oplus K \oplus M = K$

Block Cipher Recap: DES

- DES: 64-bit block cipher, 64-bit key (56-bit effective)-insecure today.
- Vulnerable to brute-force and chosen-plaintext attacks.

August	2016	The Open Source password cracking software hashcat added in DES brute force searching on general purpose GPUs. Benchmarking shows a single off the shelf Nvidia GeForce GTX 1080 Ti GPU costing US\$1000 recovers a key in an average of 15 days (full exhaustive search taking 30 days). Systems have been built with eight GTX 1080 Ti GPUs which can recover a key in an average of under 2 days. ^[25]
July	2017	A chosen-plaintext attack utilizing a rainbow table can recover the DES key for a single specific chosen plaintext <code>1122334455667788</code> in 25 seconds. A new rainbow table has to be calculated per plaintext. A limited set of rainbow tables have been made available for download. ^[26]

- Reference: https://en.wikipedia.org/wiki/Data_Encryption_Standard

Thuật ngữ: brute-force attack = tấn công vét cạn; chosen plaintext = văn bản thuần chọn trước

AES Review

- AES operates on 128-bit blocks.
- Main transformations:
 - Substitute-bytes (S-box): Non-linear substitution.
 - Shift-rows: Diffusion.
 - Mix-columns: Further diffusion.
 - Add-round-key: XOR with round key.
- Key sizes: 128, 192, 256 bits.
- Obsolete modes: Confidential only (ECB, CBC, etc): AESVisual.py
- Secure modes: AEAD (GCM and others)
- **Attacks CBC:** AES_attacks.py,

Thuật ngữ: diffusion = khuếch tán; confusion = ẩn hoá

CBC Padding Oracle: Decrypting *without* the Key (Part 1)

CBC decryption rule

$$P_k = D_K(C_k) \oplus C_{k-1}.$$

Oracle (vulnerable API): given two blocks $X \| C_k$, it returns

True iff $D_K(C_k) \oplus X$ has *valid PKCS#7* padding.

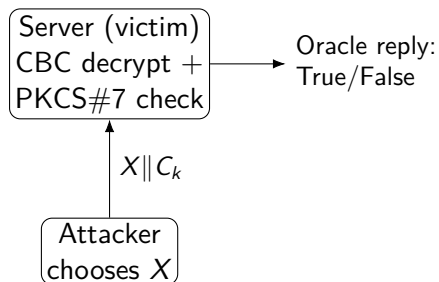
Key insight

$\underbrace{D_K(C_k)}_{\text{fixed (unknown)}} \oplus \underbrace{X}_{\text{our choice}} \Rightarrow$ we can *solve* bytes of $D_K(C_k)$ by padding guesses.

Then recover plaintext byte-by-byte:

$$P_k[i] = D_K(C_k)[i] \oplus C_{k-1}[i].$$

CBC Padding Oracle: Decrypting *without* the Key (Part 2)



Vary X until
padding valid
⇒ leak one byte

Why no key needed?

- Oracle = 1-bit side channel about padding validity.
- By *choosing* X , attacker solves $D_K(C_k)$, hence P_k .

Fix

- Use **AEAD** (e.g., GCM/EAX) — authenticate before decrypt.
- If CBC remains: *Encrypt-then-MAC*, **uniform errors**, constant-time.

Warm-up: AES and the Key Problem

- Which AES mode or algorithm do we actually use? (GCM, CCM, OCB etc.)
- Where does the AES key come from?
- How do both sides agree on the same key securely?
- What happens if an attacker intercepts or replaces the key?
- **Setup:** matplotlib, numpy and run the codes: AES_run.py

Motivation: Negotiating Secret Keys

- In symmetric cryptography, both sides must share a key.
- **Question:** how to negotiate securely without a trusted Key Distribution Center (KDC)?
- → Motivation for asymmetric cryptography.

Thuật ngữ: key negotiation = thỏa thuận khóa; KDC = trung tâm phân phối khóa

Why Asymmetric Cryptography?

- Overcomes key limitations of symmetric encryption:
 - ① **Key Distribution:** Eliminates the need for secure pre-shared keys.
 - ② **One-to-Many Communication:** One sender can encrypt for multiple receivers, even when receivers are offline.
 - ③ **Digital Signatures:** Enables authentication and non-repudiation (mutual authentication).
- Introduced by Whitfield Diffie and Martin Hellman (1976).

Thuật ngữ: asymmetric cryptography = mật mã bất đối xứng; digital signature = chữ ký số

Symmetric vs Asymmetric

	Symmetric	Asymmetric
Key type	Shared secret	Public/Private pair
Encryption/Decryption	Same key	Different keys
Speed	Fast	Slower
Security basis	Key secrecy	Mathematical trapdoor

Thuật ngữ: trapdoor = cửa bẫy; key pair = cặp khóa

Public-Key Cryptosystems

- Components:
 - 1 Plaintext, Ciphertext
 - 2 Encryption/Decryption algorithms
 - 3 Public and Private keys
- Example: Confidentiality — Encrypt with recipient's public key, decrypt with private key.

Thuật ngữ: public key = khóa công khai; private key = khóa bí mật

Secure Communication (Public-Key Encryption)

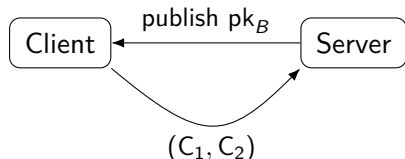
Goal: Only the intended receiver can read the message.

Abstract interface

- $(pk, sk) \leftarrow \text{KeyGen}()$
- $c \leftarrow \text{Enc}(pk, m)$
- $m \leftarrow \text{Dec}(sk, c)$

Pseudocode (hybrid encryption)

```
# Receiver Server publishes  $pk_B$ ; keeps  $sk_B$ 
  secret
(pk_B, sk_B) := KeyGen()
# Sender Client
K := Random(256 bits) # session key (symmetric)
C1 := Enc(pk_B, K) # PKE: encapsulate K
C2 := AEAD_Encrypt(K, nonce, AAD, m) # ex: AES-GCM
send (C1, nonce, AAD, C2)
# Receiver Server
K := Dec(sk_B, C1)
m := AEAD_Decrypt(K, nonce, AAD, C2)
```



- | | |
|--------------------------------|----------------------------------|
| 1) Get pk_B | 1) $K = \text{Dec}(sk_B, C_1)$ |
| 2) Generate K | 2) $m = \text{AEAD}_K^{-1}(C_2)$ |
| 3) $C_1 = \text{Enc}(pk_B, K)$ | |
| 4) $C_2 = \text{AEAD}_K(m)$ | |

Thuật ngữ: public key = khoá công khai; secret key = khoá bí mật

Authentication & Integrity (Digital Signatures)

Goal: Verify *who* sent the data and that it *was not modified*.

Abstract interface

- $(pk_A, sk_A) \leftarrow \text{KeyGen}()$
- $\sigma \leftarrow \text{Sign}(sk_A, m)$
- $\{\top, \perp\} \leftarrow \text{Verify}(pk_A, m, \sigma)$

Raw idea

```
# User Client owns  $(pk_A, sk_A)$ 
```

```
 $(pk_A, sk_A) := \text{KeyGen}()$ 
```

```
# Sign
```

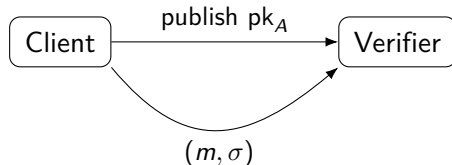
```
sigma := Sign(sk_A, m)
```

```
send (m, sigma, pk_A)
```

```
# Verify (any verifier)
```

```
ok := Verify(pk_A, m, sigma)
```

```
if ok == true: accept # authenticated & intact  
else: reject
```



1) (pk_A, sk_A)

2) $\sigma = \text{Sign}(sk_A, m)$

1) check $\text{Verify}(pk_A, m, \sigma)$

2) accept / reject

Thuật ngữ: authentication = xác thực;
non-repudiation = không thể chối bỏ

Securely Establishing a Shared Secret: Key Encapsulation Mechanism(KEM)

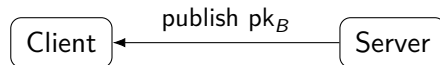
Goal: Derive a fresh symmetric key K for AEAD without prior shared secret.

KEM: One-way key delivery (e.g., Hybrid Encryption)

```
# Receiver Server
(pk_B, sk_B) := KEM.KeyGen()
send pk_B to Sender

# Sender Client
C := KEM.Encaps(pk_B, K) #encapsulate random K
send C to Receiver

# Receiver Server
K := KEM.Decaps(sk_B, C) # recover same K
# Use K with AEAD for data channel
```



$$1) C = \text{Encaps}(pk_B, K)$$

$$1) K = \text{Decaps}(sk_B, C)$$



Thuật ngữ: shared secret = bí mật chung; AEAD = mã hoá kèm xác thực

Thuật ngữ: KEM = bao gói khoá

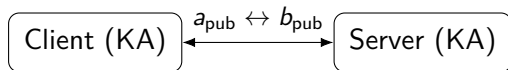
Securely Establishing a Shared Secret: Key Agreement Mechanism(KAM)

Goal: Both parties jointly derive the same symmetric key K through mutual exchange.

KAM: Mutual key derivation (e.g., Diffie–Hellman / X25519)

```
# Each side makes an ephemeral key pair
(a_pub, a_sec) := KA.KeyGen()
(b_pub, b_sec) := KA.KeyGen()
exchange a_pub <--> b_pub
```

```
# Each derives the same shared key
K := KA.Derive(a_sec, b_pub) # Client
K := KA.Derive(b_sec, a_pub) # Server
# Use K with AEAD for data channel
```



$$K = \text{Derive}(a_{\text{sec}}, b_{\text{pub}})$$

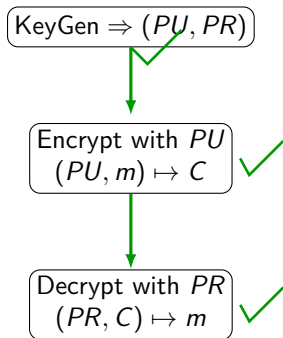
$$K = \text{Derive}(b_{\text{sec}}, a_{\text{pub}})$$

Thuật ngữ: shared secret = bí mật chung; AEAD = mã hoá kèm xác thực

Thuật ngữ: KAM = cơ chế thoả thuận khoá

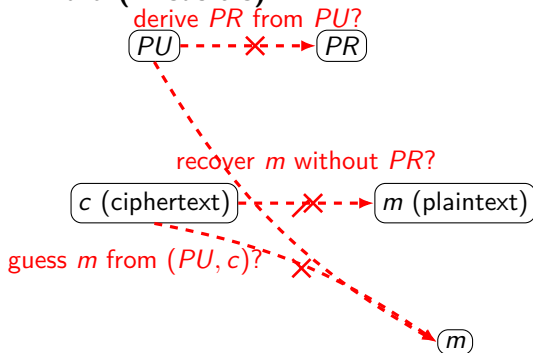
Public-Key Requirements: Easy vs. Hard

Easy (Feasible)



- feasible (efficient)
- - - infeasible (computationally hard)
- ✓ / ✗ passes / disallowed by design

Hard (Infeasible)



Thuật ngữ: infeasible = *bất khả thi tính toán*; feasible = *khả thi*

Public-Key Requirements (2/2)

- All based on **trapdoor one-way functions** f_k :

$$Y = f_k(X) \text{ easy, } X = f_k^{-1}(Y) \text{ infeasible without } k$$

- Security relies on computational hardness assumptions.

Thuật ngữ: one-way function = hàm một chiều; trapdoor = cửa bẫy

RSA: Rivest–Shamir–Adleman (1977)

- Most widely used public-key encryption.
- Based on integer factorization problem.
- Typical modulus $n = 3072$ bits.

$$C = M^e \bmod n, \quad M = C^d \bmod n$$

Thuật ngữ: factorization = phân tích thừa số nguyên tố

RSA Key Generation

- 1 Choose large primes p, q .
- 2 Compute $n = pq$, $\varphi(n) = (p - 1)(q - 1)$.
- 3 Choose public exponent e coprime with $\varphi(n)$.
- 4 Compute private key d such that $ed \equiv 1 \pmod{\varphi(n)}$.

Thuật ngữ: Euler function = hàm phi Euler

OpenSSL: RSA Keygen & Checks

Recommended: RSA 3072+ and OAEP (SHA-256). Files are PEM.

```
# 1) Generate private key (PEM, PKCS#8)
openssl genpkey -algorithm RSA \
  -pkeyopt rsa_keygen_bits:3072 \
  -out rsa_priv.pem

# 2) Derive public key (SubjectPublicKeyInfo PEM)
openssl rsa -in rsa_priv.pem -pubout -out rsa_pub.pem

# 3) Text dump (private): parameters & n,e,d shown (secret: d)
openssl pkey -in rsa_priv.pem -text -noout
# Quick consistency check (legacy rsa command)
openssl rsa -in rsa_priv.pem -check -noout

# 4) Print modulus (n) if needed
openssl rsa -in rsa_priv.pem -noout -modulus
openssl rsa -pubin -in rsa_pub.pem -noout -modulus
```

Thuật ngữ: keygen = sinh khóa; modulus n = mô-đun; inspect = kiểm tra

OpenSSL: RSA Encrypt / Decrypt (OAEP)

Use OAEP (SHA-256). Encrypt small data *or* a random session key (hybrid).

Example input

```
echo -n "hello RSA OAEP" > msg.bin
```

Encrypt with PUBLIC key (OAEP + SHA-256)

```
openssl pkeyutl -encrypt -pubin \  
-inkey rsa_pub.pem \  
-pkeyopt rsa_padding_mode:oaep \  
-pkeyopt rsa_oaep_md:sha256 \  
-pkeyopt rsa_mgf1_md:sha256 \  
-in msg.bin -out c.bin
```

Decrypt with PRIVATE key

```
openssl pkeyutl -decrypt \  
-inkey rsa_priv.pem \  
-pkeyopt rsa_padding_mode:oaep \  
-pkeyopt rsa_oaep_md:sha256 \  
-pkeyopt rsa_mgf1_md:sha256 \  
-in c.bin -out plain.bin
```

```
cat plain.bin
```

Fast Modular Exponentiation (square-and-multiply)

Task: compute $a^b \bmod n$ efficiently using binary exponent.

Binary idea

$$b = \sum_{i=0}^{k-1} b_i 2^i, \quad b_i \in \{0, 1\},$$

$$a^b \equiv \prod_{i=0}^{k-1} \left(a^{2^i}\right)^{b_i} \pmod{n}.$$

Bit-view (MSB on left)

$$b = 560 = 512 + 32 + 16$$

$$560 = \overbrace{1}^{2^9} 000 \overbrace{11}^{2^5, 2^4} 0000_2$$

Pseudocode

```

Input: a, b, n # with b >= 0
x := a mod n
res := 1
while b > 0:
    if (b mod 2) == 1:
        res := (res * x) mod n
    x := (x * x) mod n
    b := floor(b / 2)
return res

```

- At each step: *square* the base; if bit is 1, *multiply* into result.
- Complexity: $O(\log b)$ squarings and $\leq \lfloor \log_2 b \rfloor + 1$ multiplies.

square-and-multiply = *bình phương-nhân*

Worked Example: $7^{560} \bmod 561$

Setup. $b = 560 = 2^9 + 2^5 + 2^4$, $n = 561$. (We'll iterate from LSB to MSB.)

i	bit b_i	$x = 7^{2^i} \bmod 561$	res (before)	res (after)
0	0	7	1	1
1	0	49	1	1
2	0	157	1	1
3	0	526	1	1
4	1	103	1	103
5	1	511	103	460
6	0	256	460	460
7	0	460	460	460
8	0	103	460	460
9	1	511	460	1

Result. $7^{560} \equiv 1 \pmod{561}$.

Sanity check (number theory). $561 = 3 \cdot 11 \cdot 17$ là số Carmichael, với $\lambda(561) = \text{lcm}(2, 10, 16) = 80$. Nếu $\gcd(a, 561) = 1$ thì $a^{80} \equiv 1 \pmod{561}$. Vì $560 = 7 \cdot 80$, ta có $7^{560} \equiv 1 \pmod{561}$ — khớp với bảng tính.

Efficient Operation

- Public exponent e : common choice $65537 = 2^{16} + 1$.
- Small e (e.g., 3 or 17) improves performance but increases risk.
- For private key: use Chinese Remainder Theorem (CRT) to accelerate decryption.

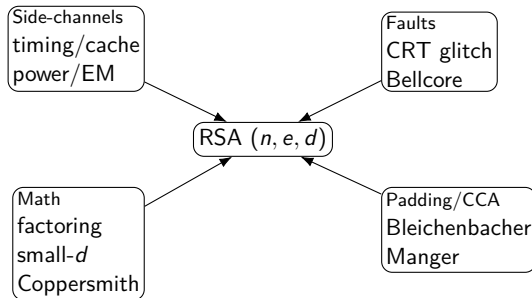
Thuật ngữ: Chinese Remainder Theorem = Định lý phần dư Trung Hoa

RSA Security Landscape

Goal: understand where RSA breaks & how to harden it.

Attack surface

- **Mathematical:** factoring n , small- d (Wiener/Boneh–Durfee), small- e broadcasts, common-modulus, Coppersmith partial info.
- **Padding/CCA:** PKCS#1 v1.5 oracles (Bleichenbacher), Manger.
- **Side-channels:** timing (Kocher 1996), cache, power/EM.
- **Faults:** CRT recombination (Bellcore), glitching.
- **Keygen/ops:** bad randomness, shared primes, weak e , multi-prime.
- **Quantum:** Shor's algorithm $\Rightarrow$ RSA broken at scale.



Thuật ngữ: attack surface = bề mặt tấn công

Mathematical Attacks (I): Factoring & Exponent Pitfalls

Factoring $n = pq$

- If n is factored $\Rightarrow$ compute $\varphi(n) \Rightarrow d = e^{-1}(\text{mod } \varphi(n))$.
- Bad keygen $\Rightarrow$ *shared primes* across certs $\Rightarrow$ gcd-collisions reveal p .

Small- d

- **Wiener (1990):** breaks when $d < n^{0.25}/3$ (continued fractions).
- **Boneh–Durfee (2000):** $d < n^{0.292}$ (lattices/Coppersmith).

Small- e misuse

- **Håstad broadcast:** same m to $\geq e$ recipients with $e = 3$ and no padding $\Rightarrow$ recover m by CRT.
- **Common-modulus:** same n reused with different e_1, e_2 on same $m \Rightarrow$ combine to get m .

Coppersmith methods

- Partial key/prime leaks (known MSBs/LSBs) or *structured* RSA moduli enable polynomial-time recovery.

Thuật ngữ: small- d = mũ giải mã nhỏ; broadcast = phát rộng

Mathematical Attacks (II): Parameters & Defenses

Safe parameters

- Modulus: ≥ 2048 bits (better: 3072) today.
- Choose $e = 65537$; avoid $e = 3$ in new systems.
- Random, independent strong primes; reject too-close primes ($|p - q|$ small).
- Avoid multi-prime RSA unless carefully justified and audited.

Operational checks

- Never reuse n across identities/systems.
- Enforce keygen health: RNG self-tests, primality with proofs or strong checks.
- Periodic scans for *shared factors* via public-key datasets (batch GCD).

Thuật ngữ: primality test = kiểm tra nguyên tố; batch GCD = GCD hàng loạt

Padding & CCA Attacks

Why raw RSA is dangerous

- Deterministic & multiplicatively homomorphic $\Rightarrow$ *malleable*.
- PKCS#1 v1.5 (historic) $\Rightarrow$ **Bleichenbacher'98** padding oracles (IND-CCA break).
- **Manger'01** (OAEP decryption oracles if mis-checked).

Modern choices

- **RSA-OAEP** for encryption (IND-CCA in the RO model).
- **RSA-PSS** for signatures (provable security; randomized salt).
- Prefer **hybrid**: RSA-OAEP encapsulates a random K , use AEAD (e.g., AES-GCM) for data.

Rule of thumb

- No custom padding. No “raw RSA”. No legacy rsaut1 with v1.5 in new designs.
- Constant-time parsing; uniform errors; side-channel-safe code paths.

Thuật ngữ: IND-CCA = bảo mật trước tấn công chọn bản mã thích ứng

Side-Channel & Fault Attacks

Timing/cache (Kocher 1996)

- Square-and-multiply leaks via data-dependent branches or table lookups.
- Cache attacks (Prime+Probe, Flush+Reload) recover key bits on shared hardware.

Power/EM analysis

- SPA/DPA/Template attacks on exponentiation or CRT sub-exponentiations.

Faults (Bellcore attack)

- Induce one faulty CRT signature s^* ; compute $\gcd(n, s - s^*)$ to recover a prime.
- Similar faults during CRT decryption also leak factors.

Defenses

- **Blinding**: randomize base before exponentiation.
- **Constant-time** big-int ops; no secret-dependent branches/indices.
- **Masking/randomized CRT exponents; redundancy checks** (recompute mod n , verify mod each prime).

Thuật ngữ: blinding = làm mù; CRT = định lý số dư TQ

Hardened RSA Decryption (Sketch)

Idea: CRT + blinding + verification to defeat timing & faults.

```
# Input: ciphertext c, private key (p, q, dp, dq, qInv)
# Output: m = c^d mod n, hardened
# 1) Blinding
r := Random in [1, n-1] with gcd(r, n) = 1
c' := (c * r^e mod n)
# 2) CRT exponentiation (constant-time)
m_p := (c' ^ dp) mod p
m_q := (c' ^ dq) mod q
h := ((m_p - m_q) * qInv) mod p
m' := m_q + q * h # recombination
# 3) Unblinding
rInv := r^{-1} mod n
m := (m' * rInv) mod n
# 4) Verification (fault countermeasure)
if (m^e mod n) != c: abort() # or restart / signal fault
return m
```

Thuật ngữ: constant-time = thời gian hằng; unblinding = gỡ mù

Key Management & Migration

Do

- Use **RSA-3072** or **RSA-2048** (min), $e = 65537$, OAEP/PSS only.
- Hardware-backed keys if possible; certified libraries; side-channel hardened.
- Rotate keys; monitor for shared primes; pin algorithm/padding in protocols.

Don't

- Don't roll your own padding or big-int; don't use $e = 3$ for new systems.
- Don't share n across entities; don't skip CRT verification.

Post-quantum plan

- Assume Shor's algorithm breaks RSA at scale. Start **hybrid** (PQ KEM + classical), plan migration to NIST PQC KEMs (e.g., Kyber) for key establishment; keep AEAD for data.

Thuật ngữ: hybrid PQ = lai lượng tử-cổ điển; rotation = xoay vòng khóa

OpenSSL Cheatsheet: Safe RSA Today

Keys (PKCS#8 + SPKI)

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa_priv.pem
openssl rsa -in rsa_priv.pem -pubout -out rsa_pub.pem
```

Encryption (OAEP-SHA256) / Decryption

```
# encrypt small data or a random K (hybrid)
openssl pkeyutl -encrypt -pubin -inkey rsa_pub.pem \
  -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 \
  -in msg.bin -out c.bin

openssl pkeyutl -decrypt -inkey rsa_priv.pem \
  -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 \
  -in c.bin -out plain.bin
```

Thuật ngữ: OAEP/PSS = đệm an toàn; SPKI = khóa công khai chuẩn

RSA: Quick Diagnosis Matrix

Symptom	Likely cause / Fix
Recover m from multiple c_i with $e = 3$	Håstad broadcast; use OAEP and $e = 65537$
Padding oracle visible	Bleichenbacher/Manger; switch to OAEP, uniform errors, constant-time parsing
CRT fault reveals factor	Missing verification; add $m^e \bmod n$ check or per-prime checks
Timing leaks on server	No blinding / data-dependent code; add blinding & constant-time big-int
Weak keys in scans	Shared primes or bad RNG; regenerate with strong RNG, batch-GCD audit

Thuật ngữ: diagnosis = chẩn đoán; audit = kiểm toán

Timing Attacks and Countermeasures

- **Constant exponentiation time:** all runs take same time.
- **Random delay:** adds unpredictability.
- **Blinding:** multiply ciphertext by random r before exponentiation.

Thuật ngữ: blinding = làm mù dữ liệu

Fault and Chosen-Ciphertext Attacks

- Fault attack: induce hardware errors to leak key.
- Chosen ciphertext attack (CCA): attacker requests decryptions to learn structure.
- Countermeasure: Optimal Asymmetric Encryption Padding (OAEP).

Thuật ngữ: OAEP = đệm mã hóa bất đối xứng tối ưu

Key Takeaways

- Asymmetric cryptography solves key distribution and signature problems.
- RSA: based on factoring — uses trapdoor one-way function.
- Attacks emphasize implementation security (timing, fault, CCA).
- OAEP and constant-time exponentiation strengthen RSA.

Next Session Preview

- Logarithm-based cryptography (P2)
- Elliptic Curve Cryptography (P3)
- Quantum-resistant systems overview

Thuật ngữ: elliptic curve = đường cong elliptic; quantum resistance = kháng lượng tử